

Chapter 8: The UNIX System Interface

Relational graph edition. Pull-shaped. Top-down.

How to use this document

Red = checkpoint. Green = practice. Blue = target. Walk edges in order.

Big picture

Chapter 7 taught you FILE* and printf/scanf. This chapter shows you what is underneath. File descriptors are the kernel's handles to open files. read() and write() are the fundamental system calls. Everything in Chapter 7 -- fopen, fprintf, fgets -- is a wrapper built on top of this layer. Understanding this layer is the difference between using C and understanding C.

Target problem

Write a simplified cat command using low-level read/write system calls. It should: open named files from command-line arguments (or read stdin if no args), copy contents to stdout, and print errors to stderr.

The Graph

Two-layer architecture (memorize this relationship)

Layer 2 (Chapter 7): FILE* → fopen/fclose → fprintf/scanf → fgets/puts
Layer 1 (This chapter): int fd → open/close → read/write → lseek
Layer 2 is built ON TOP of Layer 1. FILE* wraps a file descriptor + a user-space buffer.

Node inventory

Node	Type	Purpose
file descriptor	int	Kernel handle to an open file. Small non-negative integer
0, 1, 2	int	stdin, stdout, stderr -- always open at program start
open(path, flags, mode)	syscall	Opens file, returns fd. Flags: O_RDONLY, O_WRONLY, O_RDWR, O_CREAT
creat(path, mode)	syscall	Create/truncate file. Equivalent to open with O_WRONLY O_CREAT O_TRUNC

close(fd)	syscall	Release file descriptor back to kernel
read(fd, buf, n)	syscall	Read up to n bytes into buf. Returns bytes read, 0 at EOF, -1 on error
write(fd, buf, n)	syscall	Write n bytes from buf. Returns bytes written, -1 on error
lseek(fd, offset, origin)	syscall	Move read/write position. origin: SEEK_SET/SEEK_CUR/SEEK_END
unlink(path)	syscall	Remove directory entry (delete file when last link removed)
BUFSIZ	constant	Optimal buffer size for I/O (defined in stdio.h, typically 4096-8192)

Edge inventory

Edge	From	To	Rule
E1	open()	fd	Returns new fd on success, -1 on failure
E2	fd	read()/write()	fd selects which open file to operate on
E3	fd	close()	Releases the fd for reuse
E4	fd	lseek()	Repositions the file offset for random access
E5	read() return	loop control	Returns 0 at EOF -- use as loop termination
E6	error	stderr (fd 2)	Always write errors to fd 2, never fd 1
E7	FILE*	fd (fileno)	FILE struct contains fd + buffer + position

Zoom in -- one edge at a time

Edge 1: File descriptors -- the kernel's handles

A file descriptor is a small non-negative integer. The kernel maintains a table of open files per process. fd 0/1/2 are pre-opened as stdin/stdout/stderr. When you call `open()`, the kernel picks the lowest available fd number and returns it. When you call `close()`, that slot becomes available again.

Problem 8.1: If a program opens 3 files in sequence (and closes none), what fd numbers will they get?

Answer: 3, 4, 5. (0, 1, 2 are taken by stdin/stdout/stderr.)

Edge 2: `read()` and `write()` -- the fundamental operations

Both take: fd, pointer to buffer, byte count. Both return: actual bytes transferred. `read()` returns 0 at EOF, -1 on error. `write()` returns -1 on error. Neither guarantees transferring all requested bytes in one call -- always check the return value.

```
#include <unistd.h>
char buf[BUFSIZ];
int n;
while ((n = read(0, buf, BUFSIZ)) > 0)
    write(1, buf, n);
```

This is a complete stdin-to-stdout copier. Five lines. No FILE*, no printf. This is the lowest level of I/O in a C program.

Problem 8.2: Why do we pass `n` (actual bytes read) to `write`, not `BUFSIZ`?

Answer: The last read before EOF may return fewer than `BUFSIZ` bytes. Writing `BUFSIZ` would output garbage past the actual data.

Edge 3: `open()` and `close()`

`open(name, flags, perms)` -- flags control access mode. Common flags: `O_RDONLY` (read), `O_WRONLY` (write), `O_RDWR` (both), `O_CREAT` (create if missing), `O_TRUNC` (truncate existing). Perms (e.g., 0644) set file permissions when creating.

```
#include <fcntl.h>
int fd = open("data.txt", O_RDONLY);
if (fd == -1) {
    write(2, "cannot open\n", 12); /* error to stderr */
    return 1;
}
/* ... use fd ... */
close(fd);
```

Problem 8.3: Write code to create a new file out . txt for writing with permissions 0644.

Answer: `int fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);` or
`int fd = creat("out.txt", 0644);`

CHECKPOINT A

Without looking: what are the three pre-opened file descriptors? What does `read()` return at EOF? What is the relationship between `FILE*` and `fd`?

Edge 4: lseek() -- random access

`lseek(fd, offset, origin)` moves the read/write position. origin: `SEEK_SET` (from start), `SEEK_CUR` (from current), `SEEK_END` (from end). Returns the new position, or -1 on error. Enables random-access file I/O.

```
lseek(fd, 0L, SEEK_SET); /* rewind to start */
lseek(fd, 100L, SEEK_SET); /* jump to byte 100 */
lseek(fd, -10L, SEEK_END); /* 10 bytes before end */
```

Problem 8.4: How would you determine the size of an open file using only `lseek`?

Answer: `long size = lseek(fd, 0L, SEEK_END);` then `lseek(fd, 0L, SEEK_SET);` to rewind.

Edge 5: How FILE* wraps fd (the key insight)

A `FILE` struct typically contains: the `fd`, a pointer to a user-space buffer, the buffer size, current position in the buffer, flags (read/write/error/EOF). `fopen()` calls `open()` to get a `fd`, allocates a buffer, returns `FILE*`. `fgetc()` checks the buffer first -- only calls `read()` when the buffer is empty. This reduces system call overhead dramatically (one `read()` per `BUFSIZ` bytes instead of per byte).

```
/* Simplified FILE structure */
typedef struct {
    int fd; /* file descriptor */
    int cnt; /* characters left in buffer */
    char *ptr; /* next character position */
    char *base; /* buffer start */
    int flag; /* mode flags */
} FILE;
```

Problem 8.5: Why is buffered I/O (`FILE*`) faster than calling `read()` one byte at a time?

Answer: Each system call has overhead (context switch to kernel and back). Buffering amortizes one system call across `BUFSIZ` bytes instead of one per byte.

Edge 6: malloc/free -- the storage allocator

`malloc()` requests memory from the OS (via `sbrk` or `mmap`), manages a free list of available blocks, and returns a pointer to a block of the requested size. `free()` returns a block to the free list. The free list is a linked list of unused memory blocks, each containing a header with the block size and a pointer to the next free block.

Key rules: Always check `malloc`'s return (`NULL` = out of memory). Never use memory after `free()`. Never `free()` the same pointer twice. Never write past the allocated size.

Problem 8.6: What happens if you call `free(p)` and then read `*p`?

Answer: Undefined behavior. The memory may have been reused or unmapped. Anything can happen -- crash, garbage data, or seemingly working (the worst case).

Solving the target problem

Walk: E1 (open files from argv) + E2 (read/write loop) + E6 (errors to stderr) + E3 (close)

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

void filecopy(int fd_in, int fd_out) {
    char buf[BUFSIZ];
    int n;
    while ((n = read(fd_in, buf, BUFSIZ)) > 0)
        write(fd_out, buf, n);
}

int main(int argc, char *argv[]) {
    if (argc == 1) {
        filecopy(0, 1); /* stdin to stdout */
    } else {
        for (int i = 1; i < argc; i++) {
            int fd = open(argv[i], O_RDONLY);
            if (fd == -1) {
                char msg[] = "cannot open: ";
                write(2, msg, sizeof(msg)-1);
                write(2, argv[i], strlen(argv[i]));
                write(2, "\n", 1);
                continue;
            }
            filecopy(fd, 1);
            close(fd);
        }
        return 0;
    }
}
```

CHECKPOINT B

Modify the cat program to also print each filename as a header before its contents (write the header to stdout using write()). Report your changes.

Final exam

Problem 1: Write a function that copies a file using low-level I/O (given source and dest filenames).

Problem 2: Implement a simplified `fgets()` using only `read()` and a static buffer.

Problem 3: Write a program that prints the byte offset of every newline in a file using `lseek()` to report position.

Problem 4: Explain why closing `fd 1` and then opening a file gives that file `fd 1` (and thus captures `stdout`).

Problem 5: Write a simplified `malloc` that allocates from a static char array (arena allocator).

CHECKPOINT C

Solve at least 3 problems. Report in chat.

Answer key

1. Walk E1+E2+E3: open source O_RDONLY, open dest O_WRONLY|O_CREAT|O_TRUNC, read/write loop, close both.
2. Walk E2+E5: static buffer + position. On call, copy from buffer to user string until newline or buffer empty. Refill buffer with read() when empty.
3. Walk E2+E4: read one byte at a time (or buffer and track offset). When byte is '\n', print current file offset.
4. Walk E1: open() returns the lowest available fd. After close(1), fd 1 is free. Next open() gets fd 1. Any write(1,...) now goes to that file.
5. Static char arena[SIZE]; int pos=0; malloc returns &arena[pos] and advances pos by requested size + alignment. free() is a no-op (or tracks with a free list).